Meeting Notes: CPU Performance, Benchmarking & System Optimization

1. Overview

This session is a technical lecture/discussion on how modern computer systems actually perform versus what their advertised specifications suggest. It covers the memory hierarchy, CPU utilization, instruction sets, profiling, matrix multiplication optimization, memory/NUMA architecture, and, in the second half, a detailed, practical framework for designing rigorous, reproducible benchmarks (prompted by a student's question about benchmarking a GSoC project integrated into DeepChem).

---

2. The Memory Hierarchy & Why Cache Exists

- CPUs and GPUs are clocked in GHz but are constantly waiting on data; they are far faster than the memory feeding them.
- Speed hierarchy (fastest → slowest): **Cache → RAM → I/O (flash/secondary storage)**.
- Approximate latencies:
  - Cache access: **4–5 nanoseconds**
  - RAM access: **50–60 nanoseconds** (~10x slower)
- Trade-off: cache is small (KB–MB) but fast; RAM is large (GB) but slower. Physical/space constraints near the processor prevent cache from scaling to RAM's size.
- **Goal of system design:** deliver data at cache speed while appearing to have RAM-level capacity; this is the entire motivation behind cache hierarchies, prefetching, and predictive loading.

---

3. What Does "CPU Utilization" Actually Mean?

- Naively defined as "percentage of time the CPU is doing something," but not all executed instructions are *useful*.
- **Speculative execution:** CPUs predict branch outcomes and execute ahead. If the prediction is wrong, the CPU must discard that work and redo it; this counts as "utilization" but produces no useful output.
- **Multi-core inefficiency example:** A single-threaded Fibonacci computation only uses one core, so utilization/parallelism is poor even if the code runs "correctly."
  - Using frameworks like **OpenMP** to parallelize across cores increases utilization, but introduces the risk of out-of-order results.
  - Correctness then requires synchronization primitives: **semaphores, mutexes, critical sections**.

- **Key insight:** A CPU rated faster on paper (e.g., clock speed, FLOPS) can be outperformed by a "slower" CPU on specific workloads, because of differences in instruction sets, compiler code generation, and architecture, not just raw frequency.

---

4. Instruction Sets & Real-World Instruction Usage

- **x86** uses variable-length, complex instructions; **ARM (RISC)** uses shorter, simpler instructions.
- Empirically, when compiling and analyzing standardized benchmarks (e.g., **SPEC CPU 2017**, **SPEC CPU 2026**, **Phoronix**), fewer than **20–30 distinct instructions account for ~99% of executed code**, even though modern CPUs support hundreds to over a thousand instruction types.
- Implication: most of an instruction set's complexity is rarely used in practice, but can't be discarded since it's occasionally needed.

---

5. Profiling ML Workloads

- Never optimize code blindly: **profile first** to generate a "heat map" of where time is actually spent.
- For virtually all ML applications, the dominant cost center is **matrix multiplication (matmul)**.
- Optimizing ML performance therefore largely reduces to optimizing matmul execution.

Factors affecting matmul/cache performance:

- Whether the matrices being multiplied **fit entirely within cache**; partial fits cause major slowdowns since the full data is needed at once.
- **Page size** (how the OS divides RAM into fixed-size chunks: 2KB, 4KB, 10KB, or huge pages of 2MB–4MB):
  - **Larger pages** → fewer pages fit in cache, but benefit from **spatial locality** (e.g., streaming audio data where nearby data is likely to be accessed next).
  - **Smaller pages** → more pages fit in cache (fewer page faults), but each page carries less useful data.
  - This is a tunable trade-off, not a fixed answer.

---

6. NUMA, CCDs, and Physical Memory Layout

- In large multi-socket systems (e.g., two sockets × 192 cores, with RAM sticks split across both sides of the motherboard), **physical distance affects memory access speed**: RAM closer to a given CPU socket is faster to access than RAM further away.

- This leads to **NUMA (Non-Uniform Memory Access) node configurations** and **NPS (NUMA Per Socket)** settings that need tuning.
- **CCDs (Core Complex Dies):** groups of cores (e.g., 8 CCDs of 8 cores each in a 64-core chip) that share cache locality as a functional block, another tunable performance factor.

---

7. Cold vs. Hot Runs & System State

- A system benchmarked **immediately after boot (cold)** performs differently than one benchmarked **after repeated use (warm/hot)**, because a fresh reboot wipes out accumulated cache state.
- CPUs and OSes build predictive state over time (e.g., a system that always opens Chrome and Word on startup will load them faster than a system doing so for the first time, because of predictive caching/prefetching).
- **Prefetching:** proactively loading data into memory before it's requested, so it's ready when needed (heavily used in gaming, for example).

---

8. The "Silicon Lottery" & Measurement Tolerance

- Due to manufacturing variance (**binning**), no two chips are identical; some run faster than others even within the same model line.
- Benchmarking typically accepts a **1–2% tolerance** due to variables like temperature, humidity, power quality, cooling/thermal throttling, and silicon variance.
- **Implication for small optimizations:** if you're trying to measure a 0.5–0.6% improvement but natural system variance is already 1–2%, you must run **multiple iterations** and compute the **geometric mean** (not arithmetic mean) of results to get a reliable, reproducible score.

---

9. Designing a Proper Benchmark Suite

- No single benchmark is sufficient: one benchmark only reflects one type of workload (e.g., string compression, compute-heavy chess engines, graphics/gaming, physics/weather modeling).
- A proper suite runs **20–30 different sub-benchmarks** stressing different subsystems, often in parallel copies matching core count (e.g., 256 copies on a 256-core system).

Variables that must be standardized/decided:

- RAM allocation strategy (pinned vs. OS-managed)
- Compiler choice (GCC vs. Clang vs. vendor-specific like AMD **AOCC** or Intel **ICC**, vendor compilers often favor their own hardware)
- Optimization flags (e.g., `-O0` vs. `-O3` produce very different performance)
- Memory allocator (malloc vs. jemalloc vs. tcmalloc vs. ssmalloc, etc.)

- System/core libraries (crypto, libm, coreutils, binutils)
- OS-level kernel parameters (default vs. huge pages, VMM size, ASLR/memory randomization)
- BIOS-level settings, e.g., **power determinism vs. performance determinism** (capping TDP, e.g., 500W max, vs. letting the chip draw as much as it wants)

Power/performance trade-off:

- Systems often achieve **~90% of peak performance at ~60% of peak power**; the last 10% of performance can cost ~40% more power.
- This is why production inference GPUs often run at 70–80% capacity rather than 100%, for better total cost of ownership (TCO) and reduced thermal/burnout risk, except when benchmarking purely for marketing/competitive claims.

Hidden vendor performance caveats:

- Example: NVIDIA GPUs report separate FP16 and INT16 performance numbers, but roughly **a third of the relevant cores are shared** between the two operation types. Running both workloads simultaneously does **not** yield the naive sum of the individual benchmark numbers.

---

10. Why Writing Benchmarks Is Hard

- Benchmarks must correctly parallelize, correctly measure the *intended* variable, and reflect realistic workloads.
- Common pitfall: intending to measure CPU performance, but the benchmark is actually **I/O-bound** or **memory-bound** due to heavy disk/RAM usage in the test itself.
- Industry sticks to **standardized, consortium-developed benchmarks** (e.g., SPEC, Phoronix) to avoid subjective/self-serving comparisons between vendors.
- **Benchmark hacking:** compiler and toolchain writers who know which benchmark will be used can optimize specifically for that benchmark's code patterns (vectorization, inlining, loop unrolling), inflating benchmark scores without reflecting real-world performance. This is discouraged/penalized but still common.
- Benchmarks must evolve as new hardware capabilities emerge (e.g., ray tracing had no standardized benchmark when it was first introduced).

Referenced tools & benchmark suites:

- **Benchmark suites/workloads:** SPEC, Phoronix, LibMem, RAMSpeed/SMP, ECMWF, WRF, ASP, Quantum Espresso, CPU2K (includes n-body physics simulations, matrix multiplication, memory read/write tests)
- **Profiling/tracing tools:** strace, ptrace, Intel PIN, AMD uProf ("UUF" in transcript, likely referring to AMD's profiling tool), GCC's gprof

---

11. Q&A: Practical Benchmarking Advice (DeepChem / Hugging Face Integration Case)

A student asked how to properly benchmark a GSoC project (a library) integrated into DeepChem. Key guidance given:

1. **Control your environment:** If using cloud GPUs, use the *same provider and instance type* every time; performance can vary even for nominally identical hardware (e.g., "H100" from different providers).
2. **Change only one variable at a time.** Don't compare a new library version integrated into a new DeepChem version against an old library in an old DeepChem version; isolate whether performance changes come from the library itself or from unrelated changes in DeepChem.
3. **Standardize the build:** same compiler, same build flags, same target system, same DeepChem version; the *only* thing that should vary is the component being tested.
4. **Consider testing outside the framework too**: building/attaching the library natively (outside Hugging Face/DeepChem) helps isolate whether gains come from the library itself vs. integration-specific effects.
5. **Capture full system state per run:** use a system information/collection utility to log `lscpu`, `top`, `free`, `/proc`, etc. Confirm these are identical across runs before trusting comparisons.
6. **Run multiple iterations, compute the geometric mean**, and determine an acceptable variation threshold (commonly ~1–2%) to confirm a genuine, consistent uplift (not noise).
7. **Test across multiple input types:** random inputs, and inputs that mimic realistic user behavior; confirm improvements hold across all input sets, or clearly characterize where worst-case behavior occurs.
8. **Measure profiling/tracing overhead itself:** run with profiling on and off to quantify how much overhead the measurement process itself introduces.

On timing and planning:

- Estimate total runtime carefully: if each run takes 5–15 minutes, a full parameter sweep may take ~5 days; longer per-run times (2–5 hours) will extend this significantly.
- Preliminary results may come faster than fully consolidated, statistically confident results.
- **The hard part isn't reporting the numbers; it's explaining *why* they differ.** LLMs can help with number-crunching, but investigative work remains essential.
- **Recommended process:** Before starting, align with your team/mentors on:
  - What exactly you're trying to measure
  - What is allowed to be tweaked

- Conditions under which measurements are taken
- Send a written plan of proposed runs in advance, so any failure is attributable to execution issues rather than unclear scope.

On overhead and noise control:

- **Turn off system daemons and monitoring tools** (e.g., Grafana, Prometheus) during benchmark runs; logging/tracing overhead skews results.
- Avoid running anything else on the system during a benchmark, and verify this with a system-check utility beforehand.
- Even **SSHing into the system and running** `top` during a benchmark can introduce measurable noise; be disciplined about zero background activity during runs.

---

12. Key Takeaways

- **Raw hardware specs (clock speed, FLOPS) are not reliable predictors of real-world performance**: actual performance depends on instruction sets, compiler behavior, memory hierarchy, and system tuning.
- **Benchmarking is a deep discipline**, not a single measurement: it requires standardized methodology, multiple runs, geometric mean aggregation, and awareness of confounding variables at every layer (hardware, BIOS, OS, compiler, library, code).
- Good benchmarking engineers need a **full-stack view**, from assembly-level instruction behavior to high-level system architecture.
- For the specific GSoC/DeepChem benchmarking task: isolate one variable at a time, standardize everything else, run multiple trials, eliminate noise sources, and be prepared for the "why" behind the numbers to take longer to establish than the numbers themselves.